

STATEMENT OF WORK

ReceiptLedger — Software Project Management (SPM-458)

COURSE

SPM-458

SEMESTER

4th

INSTRUCTOR

Humera Azam

UNIVERSITY OF KARACHI • COMPUTER SCIENCE DEPARTMENT

Software Project Management

ReceiptLedger

Turning a phone photo of any paper receipt into structured, categorized spending data.

SUBMITTED BY

#	NAME	ROLL NUMBER	ROLE
001	Aazmeer Sarfaraz Faridy	B24110006002	Member
002	Arish Ahmed Khan	B24110006026	Member
003	Iman Hussain	B24110006054	Member
004	Muhammad Asad Khan	B24110006087	Team Lead
005	Syed Aun Shamsi	B24110006134	Member
006	Zainab Hashmi	B24110006162	Member

Revision: 6-sprint plan compressed to 6 weeks (1-week sprints); SPM tool switched to Jira

1 Introduction

This Statement of Work builds on the approved ReceiptLedger Project Charter, converting its objectives into a concrete technical design, sprint plan, toolchain, and measurable acceptance criteria. It also fulfills Assignment #2's Comprehensive SPM Exercise: Section 4 addresses Task 2 (PMI Knowledge Areas), Section 5 addresses Task 3 (Lifecycle Relationships), this document as a whole is Task 5 (Project Charter & SoW), and Section 11 addresses Task 6 (PMI Process Groups); Tasks 1 and 4 are addressed directly below.

Every tool and service named in this SoW is free at the scale this project needs — a deliberate constraint given the team is a six-member, single-semester, six-sprint student group with no project budget.

Revision note: the project timeline has been compressed from 12 weeks (six 2-week sprints) to 6 weeks (six 1-week sprints). This is a materially tighter schedule — accuracy checkpoints, testing passes, and buffer time are compressed accordingly, and is reflected throughout Sections 5, 9, and 12 below.

Task 1 — SPM Tool

The team's assigned SPM tool is Jira (Free Plan), used for backlog, sprint boards, and issue tracking across all six sprints — see Section 9.2 for full detail and the rationale for this choice over Trello.

Task 4 — Classic Mistakes Identified & Avoided

Classic Mistake	How This Plan Avoids It
Product: gold-plating / undefined scope	Section 2 fixes explicit in/out-of-scope boundaries carried from the Charter
Process: unrealistic schedule set before the work is understood	Section 5.3 keeps Sprints 5–6 deliberately unassigned to specific stretch features, decided from real Sprint 1–4 data instead of guessed upfront — more critical given the compressed 1-week sprint length
People: unclear ownership on a team of this size	Section 10 assigns a Primary Focus per area, with the Lead reviewing across areas rather than owning every ticket
Technology: chasing a trained ML model before validating a simpler approach	Section 6 uses rule/dictionary correction over a trained model, validated against real collected data before anything heavier is considered
Process: switching tools mid-project	Section 9.9 documents a single deliberate, early tool migration (Trello → Jira) with a clean mapping and a one-session cutover, rather than an ad-hoc switch later

2 Project Scope

Carried forward from the Charter; every design decision in this SoW is scoped against these lines.

2.1 In Scope

- Phone-camera capture (single or multi-shot) via a Flutter app.
- OCR of both printed (organized retail) and handwritten (kiriyana/small-shop) receipts.
- Normalization of item-name variants to one canonical item and unit.
- Rule-based categorization into a local spending taxonomy.
- Duplicate-receipt detection via image hashing and text similarity.
- A month-over-month, category-wise spending dashboard (Section 3).

2.2 Out of Scope

- Bank/card reconciliation, tax filing, or invoice generation.
- Categorization beyond common local household/shop keywords.

- Training a custom OCR/handwriting model from scratch.

3 Dashboard — What It Displays

Fixed directly against Objective O4 (clear spending visibility):

View	What it shows
Monthly overview	Total spend for the selected month, with delta vs. previous month
Category breakdown	Spend split by category as a chart, for the selected month
Month-over-month trend	Multi-month view per category
Receipt log	Searchable list of receipts with line items, store, date, category
Review queue	Low-confidence OCR items (Section 6) surfaced for quick manual confirmation

4 PMI Knowledge Areas — Task 2

Applied to ReceiptLedger, mapped to where each is concretely addressed in this SoW:

Knowledge Area	Applied in
Scope	Section 2
Schedule / Time	Section 5.3 (Sprint Breakdown)
Cost	Section 8 (Budget)
Quality	Section 7 (Acceptance Criteria)
Human Resource	Section 10 (Team & Roles)
Communications	Section 9 (Tooling — Slack, Jira)
Risk	Section 12 (Risks & Mitigations / Risk Register)
Stakeholder	Charter Section 06, carried forward unchanged
Integration / Procurement	Not separately applicable at this project's scale — no external vendors or contracts

5 SDLC Methodology — incl. Task 3 (Lifecycle Relationships)

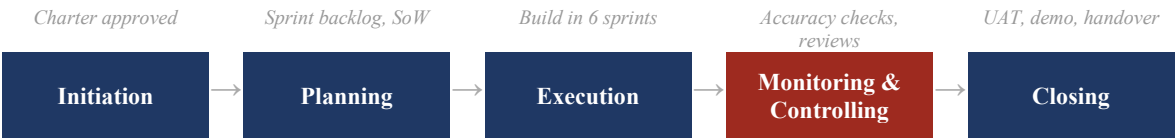
5.1 Framework: Scrum

The Charter's six-sprint cadence implies an iterative framework. Scrum fits specifically because:

Factor	Why it points to Scrum
Unproven OCR accuracy going in	Accuracy is checked mid-project (Sprint 2 into Sprint 3) and later sprints replanned around real results
Six loosely-coupled subsystems	Each can be demoed as a working slice — a natural fit for sprint-sized increments
Fixed deadline, flexible feature set	The backlog is trimmed to protect the deadline, not quality
Small, single-room team	Lightweight ceremonies suit six people without process overhead — essential given only 1 week per sprint

5.2 Project Lifecycle (Task 3)

The stages below, their dependencies, and what each hands to the next:



Monitoring & Controlling loops back into Planning/Execution each sprint (Scrum)

5.3 Sprint Breakdown — Revised: 6 sprints × 1 week = 6 weeks

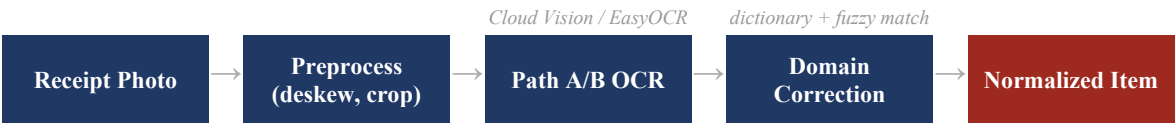
Each sprint is now one week rather than two. Exit criteria are unchanged in substance but scoped down per sprint — Sprint 1 setup and Sprint 6 polish are the tightest-margin weeks and should be protected from scope creep.

Sprint	Focus	Exit Criteria
1 (Wk 1)	Setup, capture & data collection	Repo, CI, Jira board live; capture screen working end-to-end; real receipt collection underway
2 (Wk 2)	OCR — printed-receipt path	Printed OCR integrated and accuracy-tested; checkpoint on whether Sprint 3 needs more handwriting time
3 (Wk 3)	OCR — handwritten path + normalization v1	Handwritten path integrated and tested; normalization v1 working; Sprint 4 scope adjusted from results
4 (Wk 4)	Categorization + duplicate detection	Rule-based categorizer and dedup live and tested against the accumulated sample
5 (Wk 5)	Dashboard + integration hardening	Dashboard live with Section 3's views; end-to-end flow stable
6 (Wk 6)	Testing, polish, demo prep	Bug-fix pass, UAT against Section 7, final demo and documentation

A visual Gantt-style timeline for this six-week plan is maintained in Jira's Timeline view (Section 9.7) and supplemented by a standalone chart for reporting — both show the same six one-week bars in sequence, with the OCR accuracy checkpoint flagged between Sprints 2 and 3.

6 OCR Architecture

The highest-risk part of the project: reliable extraction, especially from handwriting, without heavy compute. ReceiptLedger splits the problem into one capture flow feeding two extraction paths that reconverge at normalization, rather than one engine trying to solve everything.



OCR pipeline — printed and handwritten paths reconverge at correction. Low-confidence items are flagged to the dashboard Review Queue instead of silently accepted.

6.1 Path A — Printed Receipts

- Google Cloud Vision API (TEXT_DETECTION), per the Charter. Free tier: 1,000 units/month. Expected accuracy ~95–99% on clean printed text.
- Fallback: Tesseract 5 (self-hosted) if the free-tier quota runs out.

6.2 Path B — Handwritten Receipts

Public benchmarks put free/open handwriting OCR at roughly 70–90% word accuracy — the pipeline assumes raw output will contain errors and corrects in layers:

- Preprocessing: deskew, binarize, crop (OpenCV) before OCR runs.
- Engines: EasyOCR (primary, handles handwriting, CPU-runnable) plus Tesseract 5 as a second opinion for diffing low-confidence tokens.
- Domain correction: a rule/dictionary layer (not a second trained model) — OCR output fuzzy-matched against a vocabulary of common local items built from the team's own collected receipts; numeric price/quantity fields get stricter regex checks; anything still low-confidence is flagged to the dashboard's Review Queue (Section 3) instead of silently accepted.

7 Acceptance Criteria

Mapped directly to the Charter's SMART objectives, tested against the team's real collected sample:

Objective	Acceptance Criterion
O1 — Accurate extraction	≥90% accuracy on printed receipts; ≥70% on handwritten post-correction, both manually verified
O2 — Categorization	≥85% of items correctly categorized without manual correction
O3 — Duplicate detection	Zero false negatives on intentionally re-photographed test receipts; low false-positive rate
O4 — Spending visibility	Dashboard totals match manually-tallied totals for the full test dataset

Handwritten accuracy (70%) is set lower than printed (90%) deliberately — an honest target keeps O1 testable rather than aspirational. Under the compressed 6-week schedule, the Sprint 2→3 checkpoint is the only accuracy checkpoint before categorization/dedup work begins in Sprint 4 — see Section 12 for the risk this introduces and its mitigation.

8 Budget

Every component in this SoW runs on a free tier or is open-source and self-hosted. The only realistic costs are small and occasional, not structural:

Item	Estimate
OCR API overage (if Cloud Vision free tier is exceeded)	PKR 0–3,000 (Tesseract fallback avoids this entirely)
Mobile data for field data collection	PKR 1,500
Printing & submission binding	PKR 1,000
Contingency buffer	PKR 2,000
Estimated Total	PKR 4,500–10,000

No paid infrastructure, licenses, or subscriptions are required to reach a working, demoable product within the semester — consistent with the Charter's original budget estimate.

9 Project Tooling

9.1 Communication: Slack

- A pinned onboarding message covering channel purpose, set up on day one.
- Free tier covers the semester's message volume and history needs.

9.2 SPM Tool: Jira (Free Plan) — replaces Trello

Jira's free plan (up to 10 users) is the team's SPM tool for backlog, sprint boards, and issue tracking, replacing Trello. The switch is made specifically because this project runs Scrum, and Jira has native scrum functionality — real sprint start/complete cycles, backlog prioritization, story points, and burndown charts — where Trello had to approximate sprints with plain board lists. This matters more, not less, under the compressed 1-week sprint schedule: with less slack per sprint, the team needs an accurate, automatically-updating burndown rather than a manually-maintained list.

Why Jira over Trello for this project

Capability	Trello (previous)	Jira Free (current)
Sprints	Simulated via board lists (Backlog → Sprint 1 → ...)	Native sprint start/complete cycle, backlog view
Burndown / velocity	Not available	Built-in burndown chart and velocity tracking
Issue types	Cards only, no type hierarchy	Epic / Story / Task / Bug, with linking
Timeline view	None natively	Built-in Timeline (basic Gantt-style roadmap)
User cap (free)	10 collaborators per Workspace	10 users per site
Automation (free)	250 Butler runs/month	100 rule executions/month

What's included in Jira's free plan

- Unlimited projects, tasks, epics, and forms — no cap relevant to a 6-person team.
- Scrum and Kanban boards, one board per project, full backlog and sprint management.
- Basic Timeline (roadmap) view — usable as a lightweight Gantt chart; dates are edited per-issue rather than dragged, and it does not model complex cross-team dependencies. Sufficient at this project's scale (Section 9.7).

- Reports and dashboards, including burndown charts.
- 100 automation rule executions per month — adequate for a small team, though tighter than Trello's Butler allowance.
- 2 GB attachment storage. No cost, no time limit, no card required.

9.3 Scrum Fit

Native sprint tooling is the primary reason for the switch. The team runs real one-week sprints (Backlog → Sprint Planning → Active Sprint → Sprint Review) with story points estimated per issue and a burndown chart tracked automatically — directly supporting the Scrum framework in Section 5.1. With only five working days per sprint, the built-in burndown gives an early, honest signal if a sprint is off-track, which is harder to catch with a manually-updated Trello list.

9.4 Ticketing

Issues are typed as Epic (one per subsystem: Capture, OCR-Printed, OCR-Handwritten, Categorization, Dedup, Dashboard), Story (a demoable increment), Task, and Bug. Issues carry an assignee, sprint, story points, and labels for subsystem area. Linked issues (blocks / is blocked by) model the cross-subsystem dependencies in Section 6's OCR pipeline — e.g. Dashboard stories blocked by Categorization stories.

9.5 Collaboration

- Assignees and watchers on every issue; @mentions in comments notify teammates directly.
- Linked issues make cross-subsystem dependencies visible instead of implicit.
- Epics group each subsystem so the Lead can review progress across areas without owning every ticket (Section 10).
- Free-tier Slack integration is limited compared to paid Jira — the team checks Jira directly for sprint state rather than relying on rich Slack notifications; a daily-standup habit covers this gap, which matters more under 1-week sprints where a missed day is a larger fraction of the sprint.

9.6 Access: Desktop, Mobile, Browser

- No official Jira desktop application exists. Day-to-day use on laptop is via the browser — this works fully and needs no installation.
- Official Jira Cloud mobile apps are available for iOS (iPhone/iPad) and Android, covering issue creation, comments, board views, and push notifications.
- Net effect: no required install for desktop use; optional mobile app install for on-the-go access — useful given the compressed schedule leaves less room to be away from the board.

9.7 Visualization: Gantt / Timeline

Jira's built-in Timeline view gives a basic Gantt-style chart of epics and sprint-level work, sufficient for showing sprint sequencing in status updates and the final report. Real limits: no drag-and-drop rescheduling (dates are edited per-issue) and no advanced dependency chains. The team supplements it with a purpose-built sprint Gantt chart for the final report and any status update, showing all six one-week sprints in sequence with the Sprint 2→3 OCR checkpoint marked, while Jira's Timeline remains the working, always-current view day to day.

9.8 Risk Register

Jira's free plan has no dedicated risk-management module. Rather than adopt a separate paid tool, the team maintains a lightweight risk register as a shared document (Google Sheet or Confluence page — both free, pairing naturally with Jira/Slack), reviewed at each sprint review. The table below is this register, seeded from Section 12 and updated for the compressed timeline — each entry carries an owner, likelihood, impact, and status.

Risk ID	Risk	Likelihood	Impact	Mitigation / Owner
R1	Handwritten OCR accuracy falls short even after correction	Medium	High	Sprint 2→3 accuracy checkpoint; review-queue threshold tightened rather than blocking sprint. Owner: OCR lead.
R2	Free-tier quota exhaustion (Cloud Vision, Supabase, hosting, Codemagic)	Low	Medium	Tesseract fallback; usage monitored; iOS builds on-demand only. Owner: Cloud lead.
R3	Jira/Slack onboarding friction in Sprint 1	Medium	Low	One dedicated onboarding session for each, held before Sprint 1 — must be same-day given 1-week sprint length. Owner: Lead.
R4	Jira Timeline's manual-date-editing limits real-time schedule visibility	Low	Low	Supplement with a standalone Gantt chart for reporting (Section 9.7). Owner: Documentation lead.
R5 (new)	Compressed 1-week sprints leave insufficient buffer for scope overrun in any single sprint	High	High	Sprints 5–6 remain the flex zone (Section 5.3); daily standups to catch drift within 24h rather than at sprint end; scope cut before deadline cut. Owner: Lead.
R6 (new)	Only one OCR accuracy checkpoint (Sprint 2→3) before downstream work begins in Sprint 4	Medium	High	Categorization rules (Sprint 4) are built against a normalization interface, not raw OCR output, so a late accuracy shortfall in Sprint 3 doesn't force a Sprint 4 rebuild. Owner: OCR lead.

9.9 Migration Note

Existing Trello board content (cards, checklists, labels) is exported and re-created as Jira issues at the start of the sprint following adoption, mapped as: Trello list → Jira sprint; Trello card → Jira issue; Trello checklist → Jira sub-tasks or a Definition-of-Done description field; Trello label → Jira label. Given the compressed schedule, this migration is done in a single working session before Sprint 1 begins, not mid-project.

9.10 iOS Testing: Codemagic

Unchanged from the original SoW. Codemagic's free tier (500 build minutes/month on cloud Mac instances) builds and signs iOS apps without team-owned Mac hardware, using free Personal Team Apple ID signing — no \$99/year developer fee. Realistic limit: Mac minutes are the scarce resource — fine for periodic test builds before a demo, not continuous CI. Android stays the primary day-to-day build/test target.

10

Team & Roles

Member	Primary Focus
Muhammad Asad Khan (Lead)	Backend architecture, OCR pipeline, sprint coordination
Aazmeer Sarfaraz Faridy	Frontend and UI mockups
Arish Ahmed Khan	Cloud-based deployment
Iman Hussain	Documentation and reports
Syed Aun Shamsi	React dashboard, QA/testing
Zainab Hashmi	Data collection coordination

11

PMI Process Groups — Task 6

Each Process Group applied to ReceiptLedger's delivery, run by the whole team rather than one sub-group per phase — appropriate at this team size, where phases overlap rather than hand off cleanly:



Each Process Group is owned by the whole team, not a single sprint — Monitoring & Controlling runs alongside Executing throughout, consistent with the team's Scrum-based delivery plan. Timing reflects the 6-week compressed schedule.

Process Group	Activity, Decisions & Challenges
Initiating	Charter approved; problem, objectives, and stakeholders confirmed (pre-Sprint 1)
Planning	This SoW; Jira backlog built; architecture and tool decisions locked (Sprint 1, Week 1)
Executing	Build work across Sprints 1–5 (Weeks 1–5), per Section 5.3
Monitoring & Controlling	Accuracy checkpoint (Sprint 2→3), sprint reviews, backlog re-scoping — continuous, Sprints 2–6 (Weeks 2–6), with daily standups given the tighter per-sprint margin
Closing	UAT against Section 7, final demo, documentation handover (Sprint 6, Week 6)

12 Key Risks & Mitigations

This is the narrative summary; the working risk register with owners, likelihood, and impact ratings is maintained per Section 9.8 and reviewed each sprint.

Risk	Mitigation
Handwritten OCR accuracy falls short even after correction	Sprint 2→3 accuracy checkpoint; review-queue confidence threshold tightened rather than blocking the sprint
Free-tier quota exhaustion (Cloud Vision, Supabase, hosting, Codemagic)	Tesseract fallback for OCR; usage monitored; iOS builds run on-demand, not continuously
Jira/Slack onboarding friction in Sprint 1	One dedicated onboarding session for each before Sprint 1 begins
Compressed 1-week sprints reduce buffer for overrun	Daily standups; scope trimmed before deadline is ever moved; Sprints 5–6 remain the deliberate flex zone

13 Sign-off

This Statement of Work is submitted as the execution plan for the ReceiptLedger project, built on and consistent with the approved Project Charter, for review under SPM-458.

Humera Azam

Course Instructor, SPM-458

Signature: _____ Date: _____